1.Introduction to Modern Javascript and DOM

a.Write a javascript program to link javascript file with html page

```
project-folder/
│
├── index.html
├── style.css
└── script.js
```

1. index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Interactive Input Example</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>Welcome!</h1>
    <p>Enter your name below:</p>

    <input type="text" id="nameInput" placeholder="Enter your name" />
    <button onclick="displayGreeting()">Submit</button>

    <div id="output"></div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

2. style.css

```
body {
  font-family: 'Segoe UI', sans-serif;
  background: linear-gradient(135deg, #f3f4f6, #e0e7ff);
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  margin: 0;
}

.container {
```

```
  background-color: white;
  padding: 30px;
  border-radius: 15px;
  box-shadow: 0 0 20px rgba(0, 0, 0, 0.1);
  text-align: center;
}

input {
  padding: 10px;
  font-size: 16px;
  width: 60%;
  margin: 10px 0;
  border: 2px solid #a5b4fc;
  border-radius: 8px;
}

button {
  padding: 10px 20px;
  font-size: 16px;
  background-color: #6366f1;
  color: white;
  border: none;
  border-radius: 8px;
  cursor: pointer;
}

button:hover {
  background-color: #4f46e5;
}

#output {
  margin-top: 20px;
  font-size: 20px;
  color: #1f2937;
}
```

3. `script.js`

```
function displayGreeting() {
    const name = document.getElementById("nameInput").value.trim();
    const output = document.getElementById("output");

    if (name === "") {
        output.innerHTML = "<span style='color: red;'>Please enter a valid name!</span>";
    } else {
```

```
    output.innerHTML = ` Hello, <strong>${name}</strong>! Welcome to our site.`;
  }
}
```

b.Write a Javascript program to select the elements in html page using selectors

HTML + JavaScript Code Example

`getElementById("id")`	Selects a **single element** by its ID
`getElementsByClassName("class")`	Selects all elements with a given class
`getElementsByTagName("tag")`	Selects all elements of a given tag
`querySelector("selector")`	Selects the **first** element matching the CSS selector
`querySelectorAll("selector")`	Selects **all** elements matching the CSS selector

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Selectors Demo</title>
  <style>
    .highlight {
      color: white;
      background-color: #4ade80;
      padding: 10px;
      border-radius: 5px;
    }
  </style>
</head>
<body>

  <h1 id="mainTitle">Welcome to JavaScript Selector Demo</h1>
```

```
<p class="intro">This is the first paragraph.</p>
<p class="intro">This is the second paragraph.</p>
<button onclick="changeText()">Change Text</button>

<ul>
  <li>Apple</li>
  <li>Banana</li>
  <li class="special">Cherry</li>
</ul>

<div id="output"></div>

<script>
  function changeText() {
    // 1. Select by ID
    const title = document.getElementById("mainTitle");
    title.textContent = "JavaScript DOM Manipulation!";

    // 2. Select by Class
    const introParas = document.getElementsByClassName("intro");
    for (let para of introParas) {
      para.classList.add("highlight");
    }

    // 3. Select by Tag Name
    const listItems = document.getElementsByTagName("li");
    listItems[0].textContent = " Apple (Updated)";

    // 4. Select using querySelector
    const firstSpecial = document.querySelector(".special");
    firstSpecial.style.fontWeight = "bold";

    // 5. Select using querySelectorAll
    const allListItems = document.querySelectorAll("ul li");
    allListItems.forEach((li, index) => {
      li.style.marginBottom = "5px";
    });

    // Output confirmation
    const output = document.getElementById("output");
```

```
    output.innerHTML = "<strong>✔️ Elements selected and updated!</strong>";
  }
 </script>

</body>
</html>
```

c.Write a Javascript program to implement the event listeners

JavaScript program to implement Event Listeners in a simple and interactive HTML page.

It shows how to:

- Attach event listeners using `addEventListener()`
- Handle multiple events (`click`, `mouseover`, `keydown`)
- Update the DOM dynamically

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>Event Listener Demo</title>
 <style>
  body {
   font-family: 'Segoe UI', sans-serif;
   display: flex;
   flex-direction: column;
   align-items: center;
   padding: 40px;
   background-color: #f0f4f8;
  }

  button {
   padding: 10px 20px;
   font-size: 16px;
   background-color: #6366f1;
   color: white;
   border: none;
   border-radius: 8px;
   margin: 10px;
   cursor: pointer;
```

```
    }

    button:hover {
      background-color: #4f46e5;
    }

    #output {
      margin-top: 20px;
      font-size: 18px;
      color: #374151;
    }

    input {
      padding: 10px;
      font-size: 16px;
      border: 2px solid #a5b4fc;
      border-radius: 6px;
      margin-top: 20px;
    }
  </style>
</head>
<body>

  <h1>JavaScript Event Listeners</h1>

  <button id="clickBtn">Click Me</button>
  <button id="hoverBtn">Hover Over Me</button>

  <input type="text" id="inputBox" placeholder="Type something here..." />

  <div id="output"></div>

  <script src="script.js"></script>
</body>
</html>
```

script.js

```
// 1. Click event
const clickBtn = document.getElementById("clickBtn");
clickBtn.addEventListener("click", function () {
```

```
  document.getElementById("output").innerText = "🖱️ Button was clicked!";
});

// 2. Mouseover event
const hoverBtn = document.getElementById("hoverBtn");
hoverBtn.addEventListener("mouseover", function () {
  document.getElementById("output").innerText = "👆 You hovered over the button!";
});

// 3. Keydown event on input
const inputBox = document.getElementById("inputBox");
inputBox.addEventListener("keydown", function (event) {
  document.getElementById("output").innerText =
    `⌨️ You pressed: ${event.key}`;
});
```

Explanation

Event	Description	Trigger
`click`	Runs function when button is clicked	User clicks button
`mouseover`	Runs when mouse enters element	User hovers on button
`keydown`	Detects key press in input box	User types in input

d.write a javascript program to handle the click events for the html button elements

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Advanced Button Click Events</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>

  <h1>Advanced Button Actions</h1>

  <div id="buttonPanel">
```

```
    <button class="action-btn" data-id="1">Click Me 1</button>
    <button class="action-btn" data-id="2">Click Me 2</button>
    <button class="action-btn" data-id="3">Click Me 3</button>
  </div>

  <div id="resultArea"></div>

  <button id="resetBtn">Reset All</button>

  <script src="script.js"></script>
</body>
</html>
```

style.css

```
body {
  font-family: 'Segoe UI', sans-serif;
  background-color: #f9fafb;
  text-align: center;
  padding: 40px;
}

#buttonPanel {
  margin: 20px auto;
}

button {
  padding: 10px 25px;
  margin: 10px;
  font-size: 16px;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  background-color: #3b82f6;
  color: white;
  transition: background-color 0.3s ease;
}

button:hover {
  background-color: #2563eb;
}
```

```
#resetBtn {
  margin-top: 30px;
  background-color: #ef4444;
}

#resultArea {
  margin-top: 30px;
  font-size: 18px;
  color: #111827;
}
```

script.js

```
// Click counters for each button
const counters = {
  1: 0,
  2: 0,
  3: 0,
};

// Reference to the result display area
const resultArea = document.getElementById("resultArea");

// Function to handle button click
function handleButtonClick(buttonId) {
  counters[buttonId]++;
  const msg = `Button ${buttonId} clicked ${counters[buttonId]} time(s).`;

  // Change background color dynamically
  const button = document.querySelector(`button[data-id="${buttonId}"]`);
  const colorMap = ['#3b82f6', '#10b981', '#f59e0b', '#6366f1', '#ec4899'];
  const color = colorMap[counters[buttonId] % colorMap.length];
  button.style.backgroundColor = color;

  // Update result area
  resultArea.innerHTML = `<strong>${msg}</strong>`;
}

// Add event listeners to each button dynamically
document.querySelectorAll(".action-btn").forEach(button => {
```

```
  button.addEventListener("click", () => {
    const id = button.getAttribute("data-id");
    handleButtonClick(id);
  });
});

// Reset button functionality
document.getElementById("resetBtn").addEventListener("click", () => {
  Object.keys(counters).forEach(id => {
    counters[id] = 0;
    const button = document.querySelector(`button[data-id="${id}"]`);
    button.style.backgroundColor = "#3b82f6";
  });
  resultArea.textContent = "All counters reset.";
});
```

e.Write a JavaScript program to With three types of functions
i. Function declaration
ii. Function definition
iii. Arrow functions

functionindex.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Function Types Demo</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>JavaScript Function Types Demo</h1>

  <div class="input-section">
    <input type="number" id="num1" placeholder="Enter first number">
    <input type="number" id="num2" placeholder="Enter second number">
  </div>

  <div class="buttons">
    <button onclick="useDeclaration()">Add (Function Declaration)</button>
    <button onclick="useExpression()">Multiply (Function Expression)</button>
    <button onclick="useArrow()">Power (Arrow Function)</button>
```

```
  </div>

  <div id="result"></div>

  <script src="script.js"></script>
</body>
</html>
```

functionstyle.css

```
body {
  font-family: Arial, sans-serif;
  background: #f1f5f9;
  text-align: center;
  padding: 50px;
}

h1 {
  color: #1e293b;
}

.input-section input {
  padding: 10px;
  margin: 10px;
  width: 150px;
  font-size: 16px;
}

.buttons button {
  padding: 10px 20px;
  margin: 10px;
  font-size: 14px;
  cursor: pointer;
  background-color: #3b82f6;
  color: white;
  border: none;
  border-radius: 5px;
}

#result {
  margin-top: 30px;
```

```
  font-size: 20px;
  color: #0f172a;
}
```

Script.js

```
//  Function Declaration
function add(a, b) {
  return a + b;
}

// Function Expression (Definition using variable)
const multiply = function(a, b) {
  return a * b;
};

//  Arrow Function
const power = (a, b) => {
  return Math.pow(a, b);
};

// Helper: Get input values
function getInputs() {
  const num1 = parseFloat(document.getElementById("num1").value);
  const num2 = parseFloat(document.getElementById("num2").value);
  return { num1, num2 };
}

// Use Function Declaration
function useDeclaration() {
  const { num1, num2 } = getInputs();
  const result = add(num1, num2);
  document.getElementById("result").textContent = `Result of Addition: ${result}`;
}

// Use Function Expression
function useExpression() {
  const { num1, num2 } = getInputs();
  const result = multiply(num1, num2);
  document.getElementById("result").textContent = `Result of Multiplication: ${result}`;
}
```

```
// Use Arrow Function
function useArrow() {
  const { num1, num2 } = getInputs();
  const result = power(num1, num2);
  document.getElementById("result").textContent = `Result of Power: ${result}`;
}
```

Function Types Used

Function Type	Syntax	Used For
Function Declaration	`function add(a, b) {}`	Addition
Function Expression	`const multiply = function(a, b) {}`	Multiplication
Arrow Function	`const power = (a, b) => {}`	Exponentiation

Basics of React. js

a.Write a React program to implement a counter button using react class components.

Steps to Set Up a ReactJS Project in Visual Studio Code (VS Code)

- Install **Node.js** from https://nodejs.org
- Install **VS Code** from https://code.visualstudio.com

Step-by-Step Setup

1. **Open VS Code Terminal**
    - Use `Ctrl + ~` or go to `Terminal → New Terminal`
2. **Create a new React project using Vite or Create React App**

Option A: Using Vite (Fast & modern way – recommended)

npm create vite@latest react-counter-app -- --template react

cd react-counter-app

npm install

npm run dev

Option B: Using Create React App (classic method)

npx create-react-app react-counter-app

cd react-counter-app

npm start

Replace or create a new class component in `src/Counter.js`

React Code – Class Component Counter

`src/Counter.js`

```
import React, { Component } from 'react';

class Counter extends Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0
    };
  }
  handleIncrement = () => {
    this.setState({ count: this.state.count + 1 });
  };
  handleDecrement = () => {
    this.setState({ count: this.state.count - 1 });
  };
  handleReset = () => {
    this.setState({ count: 0 });
```

```
  };
  render() {
    return (
      <div style={styles.container}>
        <h1>React Class Counter</h1>
        <p style={styles.count}>{this.state.count}</p>
        <div>
          <button style={styles.button} onClick={this.handleIncrement}>➕ Increment</button>
          <button style={styles.button} onClick={this.handleDecrement}>➖ Decrement</button>
          <button style={styles.button} onClick={this.handleReset}>🔁 Reset</button>
        </div>
      </div>
    );
  }
}
const styles = {
  container: {
    textAlign: 'center',
    marginTop: '50px',
    fontFamily: 'Arial, sans-serif'
  },
  count: {
    fontSize: '36px',
    fontWeight: 'bold',
```

```
    color: '#2563eb'
  },
  button: {
    padding: '10px 15px',
    margin: '10px',
    fontSize: '16px',
    cursor: 'pointer'
  }
};
export default Counter;
```

Modify `src/App.js` to Use It

```
import React from 'react';
import Counter from './Counter';
function App() {
  return (
    <div>
      <Counter />
    </div>
  );
}

export default App;
```

To Run the App

In VS Code terminal:

```
npm run dev     # for Vite
# or
npm start       # for CRA
```

Open your browser at http://localhost:5173 (Vite) or http://localhost:3000 (CRA)

Concept	Description
React Class Component	Used to define stateful logic using `class`
`this.setState()`	Updates internal component state
`render()` method	Renders the component UI
Styling	Done inline using JS objects

Write a React program to implement a counter button using react functional components

Functional Component Counter in React

We'll use:

- **React Functional Component**
- **`useState()`** Hook
- Simple UI with Increment, Decrement, and Reset buttons

Folder Structure

```
react-counter-func/
├── src/
```

```
│   ├── App.js
│   └── Counter.js
├── index.html (auto-generated)
└── package.json (auto-generated)
```

src/Counter.js

```
import React, { useState } from 'react';

const Counter = () => {
  const [count, setCount] = useState(0); // state initialization
  // Handler functions
  const increment = () => setCount(count + 1);
  const decrement = () => setCount(count - 1);
  const reset = () => setCount(0);
  return (
    <div style={styles.container}>
      <h1>React Functional Counter</h1>
      <p style={styles.count}>{count}</p>
      <div>
        <button style={styles.button} onClick={increment}>➕ Increment</button>
        <button style={styles.button} onClick={decrement}>➖ Decrement</button>
        <button style={styles.button} onClick={reset}>🔁 Reset</button>
      </div>
    </div>
  );
};
```

```
// Inline CSS styles
const styles = {
  container: {
    textAlign: 'center',
    marginTop: '50px',
    fontFamily: 'Arial, sans-serif'
  },
  count: {
    fontSize: '36px',
    fontWeight: 'bold',
    color: '#1e40af'
  },
  button: {
    padding: '10px 15px',
    margin: '10px',
    fontSize: '16px',
    cursor: 'pointer'
  }
};
export default Counter;
```

src/App.js

```
import React from 'react';
import Counter from './Counter';
```

```
function App() {
  return (
    <div>
      <Counter />
    </div>
  );
}
export default App;
```

To Run This Project

npm create vite@latest react-counter-func -- --template react

cd react-counter-func

npm install

npm run dev

Then open your browser at http://localhost:5173

Concept	Description
`useState()`	React Hook to declare state in functional components
Event Handlers	Used to change state on button clicks
Functional Component	Cleaner, modern way to define React components

You don’t have to repeat all those commands for every React program.

What You Only Need to Do ONCE

You only need to run this **Vite setup process once** for a React project:

npm create vite@latest my-react-app -- --template react

cd my-react-app

npm install

npm run dev

Once you've done this:

- You now have a **React project folder**.
- Inside this folder, you can **write and test as many React programs** as you want.
- Just edit files like `App.js`, `App.css`, or create new components in `src/`.

For Future Programs

When you want to try a **new React program**, do this instead:

1. **Re-use your existing project** (e.g., `my-react-app`)
2. Replace or update code in `App.js`, or create new component files like `Counter.js`, `ClickHandler.js`, etc.
   Just run:
   npm run dev
3. if it's not already running.

You do **not** need to run `npm create vite@latest ...` again. That command is only for **creating a new project structure**.

Task	When to Run
`npm create vite@latest my-app -- --template react`	**Only once per project**

`npm install`	Only once per project
`npm run dev`	Every time you start the dev server
Change React code in `src/`	Anytime you want to test new code

Use One Project Folder for Many Examples

For example:

```
my-react-app/
└── src/
    ├── App.js        ← switch between programs here
    ├── Counter.js
    ├── ClickHandler.js
    └── styles.css
```

And in `App.js`, import whichever component you're testing:

```
import Counter from './Counter';
function App() {
  return <Counter />;
}
```

C.Write a React program to handle the button click events in functional component

React Functional Component: Button Click Events

Folder Structure

react-button-click/

```
├── src/
│   ├── App.js
│   └── ClickHandler.js
```

src/ClickHandler.js

```
import React, { useState } from 'react';

const ClickHandler = () => {
  const [message, setMessage] = useState("Click a button to see what happens!");
  // Handler for greeting
  const handleGreet = () => {
    setMessage(" Hello from React!");
    console.log("Greet button clicked");
  };
  // Handler for goodbye
  const handleBye = () => {
    setMessage("Goodbye!");
    console.log("Goodbye button clicked");
  };
  // Handler for resetting message
  const handleReset = () => {
    setMessage("Click a button to see what happens!");
    console.log("Reset button clicked");
  };
  return (
    <div style={styles.container}>
```

```
      <h2>React Button Click Handler</h2>
      <p style={styles.message}>{message}</p>
      <button style={styles.button} onClick={handleGreet}>Say Hello</button>
      <button style={styles.button} onClick={handleBye}>Say Goodbye</button>
      <button style={styles.button} onClick={handleReset}>Reset</button>
    </div>
  );
};
// Styling
const styles = {
  container: {
    textAlign: 'center',
    marginTop: '50px',
    fontFamily: 'Arial, sans-serif'
  },
  message: {
    fontSize: '18px',
    margin: '20px 0',
    color: '#1f2937'
  },
  button: {
    padding: '10px 20px',
    margin: '10px',
    fontSize: '16px',
```

```
    backgroundColor: '#3b82f6',
    color: 'white',
    border: 'none',
    borderRadius: '6px',
    cursor: 'pointer'
  }
};
export default ClickHandler;
```

src/App.js

```
import React from 'react';
import ClickHandler from './ClickHandler';
function App() {
  return (
    <div>
      <ClickHandler />
    </div>
  );
}
export default App;
```

To Run This Project

```
npm run dev
```

Then open: http://localhost:5173

d.Write a React program to conditionally render a component in the browser

We'll show/hide a message (`<Welcome />`) based on a button click using **conditional rendering**.

Step 1: Create `Welcome.js` in `src/`

📄 `src/Welcome.js`

```
import React from 'react';

const Welcome = () => {
  return (
    <div style={{ color: 'green', fontSize: '20px', marginTop: '20px' }}>
      ✅ Welcome to the conditional rendering demo!
    </div>
  );
};

export default Welcome;
```

Step 2: Modify `App.js` to use conditional rendering

`src/App.js`

```
import React, { useState } from 'react';
import Welcome from './Welcome';

function App() {
  const [show, setShow] = useState(false);
  const toggleWelcome = () => {
    setShow((prevShow) => !prevShow);
  };
  return (
    <div style={{ textAlign: 'center', marginTop: '50px', fontFamily: 'Arial' }}>
      <h1>Conditional Rendering Example</h1>
```

```
      <button
        onClick={toggleWelcome}
        style={{
          padding: '10px 20px',
          fontSize: '16px',
          backgroundColor: '#3b82f6',
          color: '#fff',
          border: 'none',
          borderRadius: '6px',
          cursor: 'pointer'
        }} >
        {show ? 'Hide' : 'Show'} Welcome Message
      </button>
      {/* Conditional rendering happens here */}
      {show && <Welcome />}
    </div>
  );
}
export default App;
```

Step 3: Run the Project

If you're still in the project folder (`my-react-app/`), just start the dev server:

npm run dev

Then open: http://localhost:5173

You'll see a **"Show Welcome Message"** button that toggles the welcome message conditionally.

Output

Action	Result
Click "Show Welcome Message"	The `<Welcome />` component appears
Click "Hide Welcome Message"	The component disappears

e.Write a React program to display text using String literals

React program that uses **string literals (template literals)** to display text dynamically inside a functional component.

App.js

```
import React from 'react';

function App() {
  const name = "Lakshmi";
  const age = 25;
  const course = "ReactJS Development";
  // Using string literal (template literal)
  const message = `Hello, my name is ${name}. I am ${age} years old and learning ${course}.`;
  return (
    <div style={styles.container}>
      <h1>Using String Literals in React</h1>
      <p style={styles.message}>{message}</p>
    </div>
  );
```

```
}
// Inline styling
const styles = {
  container: {
    textAlign: 'center',
    marginTop: '50px',
    fontFamily: 'Arial',
    backgroundColor: '#f3f4f6',
    padding: '30px',
    borderRadius: '10px',
    width: '60%',
    marginLeft: 'auto',
    marginRight: 'auto'
  },
  message: {
    fontSize: '18px',
    color: '#1f2937',
  }
};
export default App;
```

How to Run

In your React project folder:

```
npm run dev
```

Then open http://localhost:5173 in your browser.

Concept	Example
Template literals	`` `Hello, ${name}` ``
JSX rendering	`{message}` inside `<p>`
Inline styling	Object syntax like `{ fontSize: '18px' }`

Important concepts of React. js

a. Write a React program to implement a counter button using React use State hook

React Code: `CounterUseState.jsx`

```
import React, { useState } from 'react';

function CounterUseState() {
  // Declare a state variable named 'count', and a function to update it
  const [count, setCount] = useState(0);
  return (
    <div style={{ padding: '20px', textAlign: 'center' }}>
      <h2>React Counter using useState Hook</h2>
      <p style={{ fontSize: '24px' }}>Current Count: {count}</p>
      <button onClick={() => setCount(count + 1)} style={{ marginRight: '10px' }}>
        ➕ Increase
      </button>
      <button onClick={() => setCount(count - 1)} style={{ marginRight: '10px' }}>
        ➖ Decrease
```

```
      </button>
      <button onClick={() => setCount(0)}>
        🔄 Reset
      </button>
    </div>
  );
}
export default CounterUseState;
```

How to Add This to a React Project

1. Create a file: `src/CounterUseState.jsx`
2. Paste the code above
3. Use it in your `App.jsx`:

```
import React from 'react';
import CounterUseState from './CounterUseState';
function App() {
  return (
    <div>
      <CounterUseState />
    </div>
  );
}
export default App;
```

To Run the Project (if using Vite):

npm run dev

Then open http://localhost:5173 in your browser.

b. Write a React program to fetch the data from an API using React use Effect hook

React program to **fetch data from an API using the `useEffect` hook** and display it. This is a common pattern in React for fetching data when a component mounts.

React Code: `FetchData.jsx`

```
import React, { useEffect, useState } from 'react';

function FetchData() {
  const [users, setUsers] = useState([]);         // To store fetched data
  const [loading, setLoading] = useState(true);   // To show loading state
  const [error, setError] = useState(null);       // To handle errors
  useEffect(() => {
    // Fetch data from a sample API when the component mounts
    fetch('https://jsonplaceholder.typicode.com/users')
      .then((response) => {
        if (!response.ok) {
          throw new Error('Network response was not OK');
        }
        return response.json();
      })
      .then((data) => {
        setUsers(data);          // Save data to state
        setLoading(false);       // Set loading to false
      })
      .catch((err) => {
```

```
        setError(err.message);   // Handle errors
        setLoading(false);
      });
  }, []); // Empty dependency array = run only once after component mounts
  return (
    <div style={{ padding: '20px' }}>
      <h2>Fetch Data using useEffect Hook</h2>
      {loading && <p> Loading users...</p>}
      {error && <p style={{ color: 'red' }}> Error: {error}</p>}
      {!loading && !error && (
        <ul>
          {users.map((user) => (
            <li key={user.id}>
              <strong>{user.name}</strong> ({user.email})
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}
export default FetchData;
```

To Use It in Your App

In your `App.jsx`:

```
import React from 'react';

import FetchData from './FetchData';

function App() {

  return (

    <div>

      <FetchData />

    </div>

  );

}

export default App;
```

To Run This Project (If Using Vite):

npm create vite@latest react-api-fetch-app -- --template react

cd react-api-fetch-app

npm install

npm run dev

Then open http://localhost:5173

c. Write a React program with two react components sharing data using Props.

React Props Sharing Example without using **Vite**, you can set it up using **Create React App (CRA)**.

1. Create the Project

Open your terminal or command prompt and run:

npx create-react-app props-example

cd props-example

This will create the `props-example` folder with the full React setup.

2. Add Components

Now place your files like this:

```
props-example/
├── src/
│   ├── App.jsx
│   ├── ParentComponent.jsx
│   └── ChildComponent.jsx
```

3. `ChildComponent.jsx`

```
// src/ChildComponent.jsx
import React from 'react';
function ChildComponent(props) {
  return (
    <div style={{ border: '1px solid #ccc', padding: '10px', marginTop: '10px' }}>
      <h3>Child Component</h3>
      <p>Name: <strong>{props.name}</strong></p>
      <p>Age: <strong>{props.age}</strong></p>
    </div>
  );
}
export default ChildComponent;
```

4. `ParentComponent.jsx`

```
// src/ParentComponent.jsx
import React from 'react';
import ChildComponent from './ChildComponent';
```

```
function ParentComponent() {
  const userName = 'Lakshmi';
  const userAge = 22;
  return (
    <div style={{ padding: '20px' }}>
      <h2>Parent Component</h2>
      <p>This component shares data with its child.</p>
      <ChildComponent name={userName} age={userAge} />
    </div>
  );
}

export default ParentComponent;
```

5. `App.jsx`

```
// src/App.jsx
import React from 'react';
import ParentComponent from './ParentComponent';
function App() {
  return (
    <div>
      <ParentComponent />
    </div>
  );
}
```

export default App;

6. Update `index.js` to Use `App.jsx`

If using `.jsx` instead of `.js`, make sure `index.js` points to it.

// src/index.js

import React from 'react';

import ReactDOM from 'react-dom/client';

import App from './App';

import './index.css'; // Optional if you want to style

const root = ReactDOM.createRoot(document.getElementById('root'));

root.render(<App />);

7. Run the Project

In the terminal:

npm start

Then open:
`http://localhost:3000`

d. Write a React program to implement the forms in react

React program to **implement a form using React functional components** with `useState`.
This program includes:

A form with fields for name, email, and message
State handling for input values
Form submission with validation and display

Steps to Set Up and Run the Project Using Create React App (No Vite)

1. Create the React Project

npx create-react-app react-map-rendering

cd react-map-rendering

📁 **2. Create a new file: `StudentList.js` in `src/`**

`src/StudentList.js`

```
import React from 'react';

function StudentList() {
  const students = [
    { id: 1, name: 'Lakshmi', course: 'Computer Science' },
    { id: 2, name: 'Rahul', course: 'Electronics' },
    { id: 3, name: 'Aisha', course: 'Mechanical' },
    { id: 4, name: 'Vikram', course: 'Civil' }
  ];
  return (
    <div style={{ padding: '20px' }}>
      <h2> Student List</h2>
      <ul style={{ listStyleType: 'none', paddingLeft: 0 }}>
        {students.map((student) => (
          <li
            key={student.id}
            style={{
              background: '#f4f4f4',
              margin: '10px 0',
              padding: '10px',
              borderRadius: '8px'
```

```
          }} >
            <strong>{student.name}</strong> — {student.course}
          </li>
        ))}
      </ul>
    </div>
  );
}
export default StudentList;
```

3. Modify `App.js` in `src/`

`src/App.js`

```
import React from 'react';
import StudentList from './StudentList';
function App() {
  return (
    <div className="App">
      <StudentList />
    </div>
  );
}
export default App;
```

4. Run the App

npm start

This will open the project at http://localhost:3000

e. Write a React program to implement the iterative rendering using map() function.

React program that demonstrates **iterative rendering using the `map()` function**. This example displays a list of users inside a styled table.

1. Rendering Cards using `map()`

`UserCards.jsx`

```
import React from 'react';

function UserCards() {
  const users = [
    { id: 1, name: 'Lakshmi', age: 22 },
    { id: 2, name: 'Raj', age: 25 },
    { id: 3, name: 'Priya', age: 28 },
    { id: 4, name: 'Anil', age: 30 },
  ];
  const cardStyle = {
    border: '1px solid #ccc',
    borderRadius: '8px',
    padding: '16px',
    marginBottom: '12px',
    boxShadow: '0 2px 6px rgba(0, 0, 0, 0.1)',
    backgroundColor: '#f9f9f9',
    width: '250px'
  };
```

```
  return (
    <div style={{ padding: '20px' }}>
      <h2>User Cards</h2>
      <div style={{ display: 'flex', gap: '16px', flexWrap: 'wrap' }}>
        {users.map(user => (
          <div key={user.id} style={cardStyle}>
            <h3>{user.name}</h3>
            <p>Age: {user.age}</p>
            <p>ID: {user.id}</p>
          </div>
        ))}
      </div>
    </div>
  );
}
export default UserCards;
```

2. Conditional Styling in Rows using `map()`

`UserTableConditional.jsx`

```
import React from 'react';
function UserTableConditional() {
  const users = [
    { id: 1, name: 'Lakshmi', age: 22 },
    { id: 2, name: 'Raj', age: 35 },
    { id: 3, name: 'Priya', age: 29 },
```

```
  { id: 4, name: 'Anil', age: 19 },
];
return (
  <div style={{ padding: '20px' }}>
    <h2>User Table with Conditional Row Highlight</h2>
    <table border="1" cellPadding="10" style={{ width: '100%', borderCollapse: 'collapse' }}>
      <thead>
        <tr style={{ backgroundColor: '#ddd' }}>
          <th>ID</th>
          <th>Name</th>
          <th>Age</th>
          <th>Status</th>
        </tr>
      </thead>
      <tbody>
        {users.map(user => {
          const highlightStyle = {
            backgroundColor: user.age > 30 ? '#ffeeba' : '#e2f0d9'
          };
          return (
            <tr key={user.id} style={highlightStyle}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.age}</td>
```

```
              <td>{user.age > 30 ? 'Senior' : 'Young'}</td>
            </tr>
          );
        })}
      </tbody>
    </table>
  </div>
 );
}
export default UserTableConditional;
```

3. Update `App.jsx` to render these:

```
import React from 'react';
import UserCards from './UserCards';
import UserTableConditional from './UserTableConditional';
function App() {
 return (
  <div>
   <UserCards />
   <UserTableConditional />
  </div>
 );
}
export default App;
```

How to Run Without Vite (CRA method):

1. Run this in your terminal:

npx create-react-app my-map-examples

cd my-map-examples

2. Replace `src` folder contents with the files above.

3. Run:

npm start

Your project will open at: `http://localhost:3000`

Output

A styled table rendered with:

ID	Name	Age
1	Lakshmi	22
2	Raj	25
3	Priya	28
4	Anil	30

Introduction to Git and GitHub

What is Git?

Git is a **version control system** that allows developers to track and manage changes to code collaboratively.

What is GitHub?

GitHub is a **web-based hosting service** for Git repositories. It enables collaboration, pull requests, issue tracking, and more.

a. Setup

1. Install Git on Local Machine

➤ Windows:

- Download Git from: https://git-scm.com/download/win
- Run the installer with default settings.

2. Configure Git (User Name and Email)

Open a terminal or Git Bash and run the following commands:

git config --global user.name "Your Full Name"

git config --global user.email "your.email@example.com"

You can verify configuration using:

git config --global --list

Create GitHub Account and Generate Personal Access Token (PAT)

Create GitHub Account:

1. Visit https://github.com
2. Click **Sign up**, provide username, email, and password.
3. Verify your email to activate the account.

Generate Personal Access Token (for GitHub authentication):

GitHub now requires PATs for HTTPS authentication instead of passwords.

1. Go to https://github.com/settings/tokens
2. Click **"Generate new token"** → **"Generate new token (classic)"**

3. Provide a name and set expiration (recommended: 90 days or less).
4. Select scopes (for most basic usage, enable `repo`, `workflow`, and `read:org`).
5. Click **Generate token**
6. **Copy the token** and store it safely (you won't see it again).

Optional: Use PAT for Git Push (HTTPS method)

When pushing code to GitHub:

- Username: your GitHub username
- Password: **paste your PAT**

Or use this command to cache credentials:

```
git config --global credential.helper cache
```

Or store them securely:

```
git config --global credential.helper store
```

b.Basic Git Workflow

o Create a local repository using git init

o Create and add files → git add .

o Commit files → git commit -m "Initial commit"

o Connect to GitHub remote → git remote add origin <repo_url>

o Push to GitHub → git push -u origin main

Basic Git Workflow

Create a Local Repository

```
git init
```

This initializes a new Git repository in your current folder. It creates a hidden `.git` directory.

Create and Add Files

Create your project files (e.g., `index.html`, `app.js`, etc.).

Then, add them to the staging area:

```
git add .
```

This stages **all** changes (new, modified, deleted files).

Commit the Changes

```
git commit -m "Initial commit"
```

This saves a snapshot of your staged changes. The message should briefly describe the changes.

Connect to GitHub Remote Repository

First, create a repository on GitHub (without README or license if you already have local files).

Then link your local repo to the GitHub remote:

```
git remote add origin https://github.com/your-username/your-repo-name.git
```

Verify it:

```
git remote -v
```

Push to GitHub

If your main branch is called `main`, push it like this:

```
git push -u origin main
```

> The `-u` flag sets `origin/main` as the default upstream for future pushes and pulls.

Git Workflow with Example

Let's assume you're building a website and the project folder is named `my-website`.

1. Create a Local Repository

```
mkdir my-website
```

```
cd my-website
```

git init

Initializes an empty Git repository in the `my-website` folder.

2. Create and Add Files

➢ Using terminal (Linux/Mac/Windows Git Bash):

```
echo "<h1>Hello Git</h1>" > index.html
```

```
git add .
```

This creates an `index.html` file and stages it for commit.

You can create files using terminal commands or any text/code editor.

```
echo "Hello Git" > hello.txt
```

Or use:

```
touch index.html
```

➢ Using Notepad (Windows):

```
notepad index.html
```

Add some HTML content:

```
<!DOCTYPE html>
<html>
<head><title>Git Project</title></head>
<body>Hello Git!</body>
</html>
```

Save and close the file.

3. Track the File with Git

Check file status:

```
git status
```

You'll see something like:

Untracked files:

index.html

Now add it to Git's staging area:

```
git add index.html
```

Now Git is tracking changes to this file.

4. Commit the File

```
git commit -m "Initial commit: added index.html"
```

This saves a snapshot of your file into the Git history.

Example: Create Multiple Files

```
echo "# Git Demo" > README.md
touch style.css
touch app.js
git add .
git commit -m "Added README, CSS, and JS files"
```

Tip: Ignore Files with `.gitignore`

If there are files you **don't** want Git to track (e.g., logs, `.env`), create a `.gitignore` file:

```
echo "node_modules/" > .gitignore
```

3. Commit Files

```
git commit -m "Initial commit: added index.html"
```

This creates your first commit in the local repository.

4. Connect to GitHub Remote

1. Create a new repository on GitHub:
   e.g., `https://github.com/your-username/my-website.git`
   (Don't add a README if you've already initialized locally)
2. Connect it:

git remote add origin https://github.com/your-username/my-website.git

This links your local repo to the GitHub remote repo.

Push to GitHub

git branch -M main

git push -u origin main

Pushes your code to GitHub on the `main` branch. The `-u` sets it as the upstream.

c. Branching and Collaboration

o Create a branch → git checkout -b feature1

o Merge branch to main → git merge feature1

o Resolve merge conflicts (guided)

Create a New Branch

git checkout -b feature1

This creates a new branch named `feature1` and switches to it.

Merge Branch to Main

First, go back to the `main` branch:

git checkout main

Then merge your feature branch:

git merge feature1

Resolve Merge Conflicts (Guided)

If both `main` and `feature1` edited the same part of a file, Git shows a **conflict** like this:

<<<<<<< HEAD

Current code in main

=======

Code from feature1

>>>>>>> feature1

Steps to Resolve:

1. Open the file and **manually edit** to keep the correct version.
2. Remove the `<<<<<<<`, `=======`, `>>>>>>>` markers.
3. After resolving:

```
git add conflicted-file.txt
```

4. Then complete the merge:

```
git commit -m "Resolved merge conflict in conflicted-file.txt"
```

Branching and Collaboration Example

Create a Branch (feature1)

```
git checkout -b feature1
```

Now, let's say you modify the `index.html` file:

```
<!-- New content in feature1 branch -->
<h1>Hello from Feature Branch</h1>
```

Then commit:

```
git add index.html
git commit -m "Updated index.html in feature1 branch"
```

Merge Branch to Main

Switch to `main` branch:

```
git checkout main
```

Now merge:

git merge feature1

If no conflicts, the feature will be merged into main.

Resolve Merge Conflicts (Example)

Let's say in `main`, you also edited `index.html`:

<!-- Main branch version -->

<h1>Hello from Main Branch</h1>

Then try merging `feature1`, Git will show a conflict like:

<<<<<<< HEAD

<h1>Hello from Main Branch</h1>

=======

<h1>Hello from Feature Branch</h1>

>>>>>>> feature1

Resolve It:

Manually choose what you want:

<h1>Hello from Both Branches</h1>

Then:

git add index.html

git commit -m "Resolved merge conflict in index.html"

Upload React Project to GitHub

o Create a new React app using npx create-react-app myapp

o Initialize a git repo and push to GitHub

o Use .gitignore to exclude node_modules

o Create multiple branches: feature/navbar, feature/form

o Practice merge and pull requests (can use GitHub GUI)

Upload React Project to GitHub

Step 1: Create a New React App

npx create-react-app myapp

cd myapp

> This sets up a new React project in the `myapp` folder.

Step 2: Initialize a Git Repository

git init

git add .

git commit -m "Initial commit - React app created"

Step 3: Add `.gitignore`

> `create-react-app` already includes a `.gitignore` file with `node_modules/` ignored.

To verify:

cat .gitignore

Make sure `node_modules/` is listed — this keeps your repo clean and fast.

Step 4: Push to GitHub

1. Create a **new GitHub repo** (don't initialize with README or `.gitignore`).
2. Add the remote origin:

git remote add origin https://github.com/your-username/myapp.git

3. Push the code to GitHub:

git branch -M main

git push -u origin main

Work with Branches

Create Feature Branches

```
git checkout -b feature/navbar

# Make changes to add a navbar component

git add .

git commit -m "Add Navbar component"

git push -u origin feature/navbar

git checkout -b feature/form

# Create a form component

git add .

git commit -m "Add Form component"

git push -u origin feature/form
```

Practice Merge and Pull Requests

Option 1: Use GitHub GUI (Recommended)

1. Go to the GitHub repo page.
2. Click **“Compare & pull request”** for `feature/navbar`.
3. Review code and **create pull request**.
4. Once approved, click **“Merge pull request”**, then **“Confirm merge”**.
5. Repeat the same for `feature/form`.

Option 2: Use Git CLI

Merge locally:

```
git checkout main

git pull origin main  # get latest if needed

git merge feature/navbar

git push origin main
```

Introduction to Node. js and Express. js

a. Write a program to implement the _hello world‘ message in the route through the browser using Express

To initialize and install **Node.js** and **Express**, follow these steps carefully:

- Ensure **Node.js** and **npm** are installed.Check by running:

```
node -v
npm -v
```

If not installed, download from: https://nodejs.org/

Step 1: Initialize Node.js Project

Create a new folder for your project and navigate into it:

```
mkdir myapp
cd myapp
```

Then initialize a Node.js project:

```
npm init -y
```

This will create a `package.json` file.

Step 2: Install Express

Now install the **Express** framework via npm:

```
npm install express
```

This creates a `node_modules` folder and adds Express to your dependencies in `package.json`.

Your folder structure should now look like:

```
myapp/
├── node_modules/
├── package.json
```

└── (you will later add) app.js

Step 3: Create Your First Express App

Create a file called `app.js`:

```
// app.js

const express = require('express');

const app = express();

const PORT = 3000;

app.get('/', (req, res) => {

  res.send('Hello World');

});

app.listen(PORT, () => {

  console.log(`Server is running at http://localhost:${PORT}`);

});
```

Step 4: Run Your App

In your terminal, run:

```
node app.js
```

You'll see:

Server is running at http://localhost:3000

Visit `http://localhost:3000` in your browser — and you'll see:

Hello World

: **Step-by-Step Guide (Using VS Code)**

1. **Install Node.js** from https://nodejs.org/
2. **Install Visual Studio Code (VS Code)** from https://code.visualstudio.com/
3. Optional: Install **Git Bash** if you're on Windows, or use the VS Code Terminal

Step 1: Create a Project Folder

1. Open **VS Code**
2. Click on `File` → `Open Folder...`
3. Create a new folder (e.g., `myapp`) and open it

Step 2: Open Terminal in VS Code

- Go to `Terminal` → `New Terminal`

Step 3: Initialize a Node.js Project

Run the following command in the terminal:

npm init -y

This creates a `package.json` file automatically.

Step 4: Install Express

Install Express using npm:npm install express

This will add `express` to the `node_modules` folder and update `package.json`.

Step 5: Create `app.js` File

In your project folder:

1. Right-click → `New File`
2. Name it: `app.js`
3. Paste the following code:

```
const express = require('express');

const app = express();

const PORT = 3000;

app.get('/', (req, res) => {

  res.send('Hello World from Express!');

});
```

app.listen(PORT, () => {

  console.log(`Server is running at http://localhost:${PORT}`);

});

Step 6: Run the App

In the VS Code terminal, run:

node app.js

If everything is correct, you'll see:

Server is running at http://localhost:3000

Open your browser and visit: **http://localhost:3000**
You should see:`Hello World from Express!`

Step-by-Step Express.js Website with Multiple Routes

my-website/

├── package.json

├── app.js

└── public/

    └── styles.css

Step 1: Initialize and Install Express

In terminal (inside `my-website` folder):

npm init -y

npm install express

Step 2: Create `app.js`

const express = require('express');

const app = express();

const PORT = 3000;

// Serve static files from the "public" directory

```
app.use(express.static('public'));

// Routes

app.get('/', (req, res) => {

  res.send(`

    <h1>Welcome to My Website</h1>

    <p><a href="/about">About</a> | <a href="/services">Services</a> | <a
href="/contact">Contact</a></p>

  `);

});

app.get('/about', (req, res) => {

  res.send(`

    <h1>About Us</h1>

    <p>This is a simple Express website.</p>

    <a href="/">Back to Home</a>

  `);

});

app.get('/services', (req, res) => {

  res.send(`

    <h1>Our Services</h1>

    <ul>

      <li>Web Development</li>

      <li>SEO</li>

      <li>Hosting</li>

    </ul>

    <a href="/">Back to Home</a>
```

```
  `);
});
app.get('/contact', (req, res) => {
  res.send(`
    <h1>Contact Us</h1>
    <form>
      <label>Name: <input type="text" /></label><br />
      <label>Email: <input type="email" /></label><br />
      <input type="submit" value="Send" />
    </form>
    <a href="/">Back to Home</a>
  `);
});
// Start the server
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

Step 3: Add `styles.css` in `public/styles.css`

Create a folder `public/` and inside it, create a file `styles.css`:

```
body {
  font-family: Arial, sans-serif;
  margin: 2rem;
  background-color: #f2f2f2;
}
```

```
h1 {
  color: #333;
}
a {
  color: blue;
  text-decoration: none;
}
```

Now your app will serve `styles.css` if linked in HTML using `<link rel="stylesheet" href="/styles.css">`.

Step 4: Run Your Website

```
node app.js
```

Visit:
http://localhost:3000

Write a program to print the _hello world‘ in the browser console using Express. js.

Create `app.js`

```
const express = require('express');
const app = express();
const PORT = 3000;
// Serve static files from "public" folder
app.use(express.static('public'));
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

Create public/index.html

Inside a folder named public, create index.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>Hello Console</title>
</head>
<body>
  <h1>Hello from Express!</h1>
  <script>
    console.log("Hello World from Express!");
  </script>
</body>
</html>
```

Run Your App

```
node app.js
```

http://localhost:3000

Write a program to implement the CRUD operations using Express. Js

Project Structure:

```
crud-app/
├── app.js
└── package.json
```

Step 1: Initialize Project and Install Express

```
mkdir crud-app
cd crud-app
```

Step 2: Create `app.js`

```
const express = require('express');

const path = require('path');

const app = express();

const PORT = 3000;

app.use(express.urlencoded({ extended: true }));

app.use(express.json());

app.use(express.static(path.join(__dirname, 'public')));

// In-memory student data

let students = [

  { id: 1, name: 'Alice' },

  { id: 2, name: 'Bob' }

];

// Serve home HTML

app.get('/', (req, res) => {

  res.sendFile(path.join(__dirname, 'public', 'index.html'));

});

// Get all students

app.get('/students', (req, res) => {

  res.json(students);

});

// Get one student by ID

app.get('/students/:id', (req, res) => {

  const student = students.find(s => s.id === parseInt(req.params.id));
```

```
  if (!student) return res.status(404).send('Student not found');
  res.json(student);
});
// Add student
app.post('/students', (req, res) => {
  const newStudent = {
    id: students.length + 1,
    name: req.body.name
  };
  students.push(newStudent);
  res.send(`Student "${newStudent.name}" added successfully.`);
});
// Update student
app.put('/students/:id', (req, res) => {
  const student = students.find(s => s.id === parseInt(req.params.id));
  if (!student) return res.status(404).send('Student not found');
  student.name = req.body.name;
  res.send(`Student ${student.id} updated to "${student.name}"`);
});
// Delete student
app.delete('/students/:id', (req, res) => {
  const index = students.findIndex(s => s.id === parseInt(req.params.id));
  if (index === -1) return res.status(404).send('Student not found');
  const deleted = students.splice(index, 1);
```

```
  res.send(`Student "${deleted[0].name}" deleted.`);
});
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

public/index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Create Student</title>
</head>
<body>
  <h2>Create New Student</h2>
  <form action="/students" method="POST">
    <input name="name" placeholder="Enter name" required />
    <button type="submit">Add Student</button>
  </form>
  <p>To view students, go to <a href="/students" target="_blank">/students</a></p>
</body>
</html>
```

Step 3: Run Your App

```
node app.js
```

API Testing (with Postman or cURL):

Method	Endpoint	Description
GET	`/students`	Get all students
GET	`/students/:id`	Get one student
POST	`/students`	Add a new student
PUT	`/students/:id`	Update existing student
DELETE	`/students/:id`	Delete a student

How to Run

1. Open terminal in the project folder.
2. Run: npm install and npm start

Open browser:

- Go to `http://localhost:3000` to open the form.
- Go to `http://localhost:3000/students` to view all students.

Write a program to establish the connection between API and Database using Express – My SQL driver

1. Install Node.js & npm

Download & install from: https://nodejs.org/

2. Set up your project

```
mkdir express-mysql-api

cd express-mysql-api

npm init -y

npm install express mysql2
```

MySQL Table

Make sure you have MySQL running. Create a database and table using a MySQL client like `MySQL Workbench`, `phpMyAdmin`, or CLI.

```
CREATE DATABASE testdb;

USE testdb;

CREATE TABLE users (

  id INT AUTO_INCREMENT PRIMARY KEY,

  name VARCHAR(100),

  email VARCHAR(100)

);
```

`app.js` – Express API connected with MySQL

```
const express = require('express');

const mysql = require('mysql2');

const app = express();

const PORT = 3000;

app.use(express.json());

// MySQL connection

const db = mysql.createConnection({

  host: 'localhost',

  user: 'root',
```

```
  password: '',      // your MySQL password
  database: 'testdb'
});
// Connect to MySQL
db.connect(err => {
  if (err) {
    console.error(' Database connection failed:', err);
    return;
  }
  console.log('Connected to MySQL Database');
});
// GET all users
app.get('/users', (req, res) => {
  db.query('SELECT * FROM users', (err, results) => {
    if (err) return res.status(500).send(err);
    res.json(results);
  });
});
// POST new user
app.post('/users', (req, res) => {
  const { name, email } = req.body;
  const sql = 'INSERT INTO users (name, email) VALUES (?, ?)';
  db.query(sql, [name, email], (err, result) => {
    if (err) return res.status(500).send(err);
```

```
    res.send('User added successfully!');
  });
});
// GET user by ID
app.get('/users/:id', (req, res) => {
  const sql = 'SELECT * FROM users WHERE id = ?';
  db.query(sql, [req.params.id], (err, result) => {
    if (err) return res.status(500).send(err);
    if (result.length === 0) return res.status(404).send('User not found');
    res.json(result[0]);
  });
});
// PUT update user
app.put('/users/:id', (req, res) => {
  const { name, email } = req.body;
  const sql = 'UPDATE users SET name = ?, email = ? WHERE id = ?';
  db.query(sql, [name, email, req.params.id], (err, result) => {
    if (err) return res.status(500).send(err);
    res.send('User updated successfully!');
  });
});
// DELETE user
app.delete('/users/:id', (req, res) => {
  const sql = 'DELETE FROM users WHERE id = ?';
```

```
  db.query(sql, [req.params.id], (err, result) => {
    if (err) return res.status(500).send(err);
    res.send('User deleted successfully!');
  });
});
// Start server
app.listen(PORT, () => {
  console.log(` Server running at http://localhost:${PORT}`);
});
```

Step 1: Setup Project

1. **Open VS Code**.
2. **Open Folder** → `File > Open Folder` → Select your project folder (`express-mysql-api`).
3. **Create a file** named `app.js`.
4. Paste the full Express + MySQL code (shared above) into `app.js`.

Step 2: Install Required Packages

Open a terminal in VS Code (`Ctrl + ~`) and run:

```
npm init -y
npm install express mysql2
```

Step 3: Start MySQL & Create Database/Table

Use MySQL Workbench or CLI:

```
CREATE DATABASE testdb;
USE testdb;
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```
  name VARCHAR(100),
  email VARCHAR(100)
);
```

Step 4: Run the Server

In the terminal (inside the project folder):

```
node app.js
```

Output:

```
Connected to MySQL Database
 Server running at http://localhost:3000
```

Using Browser

Only supports `GET` requests:

- Open: `http://localhost:3000/users`
- Open: `http://localhost:3000/users/1`

Introduction to My SQL

Write a program to create a Database and table inside that database using My SQL Command line client

step-by-step installation instructions for MySQL Command Line Client

Download MySQL Installer

- Visit: https://dev.mysql.com/downloads/installer/
- Choose **MySQL Installer for Windows** (select the full version).

2. Run the Installer

- Double-click the `.msi` installer file.
- Choose "**Developer Default**" installation.

3. Install Required Components

Make sure to select:

- **MySQL Server**
- **MySQL Command Line Client**
- **MySQL Workbench** (optional GUI)

Click **Next** → **Execute** to install all selected tools.

4. Configure MySQL Server

- Set your **root password**.
- Choose server config options (default is fine).
- Finish configuration and apply settings.

5. Launch MySQL Command Line Client

- Go to Start Menu → Search: **MySQL Command Line Client**
- Open it → Enter your root password to log in.

If installed, open it using:

```
mysql -u root -p
```

It will prompt for your MySQL password. Enter it to proceed.

Create a New Database

```
CREATE DATABASE college_db;
```

Output:

```
Query OK, 1 row affected (0.01 sec)
```

3. Use the Database

```
USE college_db;
```

Output:

```
Database changed
```

4. Create a Table Inside the Database

```
CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    name VARCHAR(100),
    email VARCHAR(100),
    course VARCHAR(50)
);
```

5. Verify the Table

```
SHOW TABLES;
```

6. View Table Structure

```
DESCRIBE students;
```

b.Write a My SQL queries to create table, and insert the data, update the data in the table

1. Create a Database

```
CREATE DATABASE CollegeDB;
```

2. Use the Database

```
USE CollegeDB;
```

3. Create a Table

Example: Students table with ID, Name, Age, and Department

```
CREATE TABLE Students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    age INT,
    department VARCHAR(50)
);
```

4. Insert Data into the Table

```
INSERT INTO Students (name, age, department)
VALUES
```

```
('Alice', 20, 'Computer Science'),
('Bob', 21, 'Electrical'),
('Charlie', 22, 'Mechanical');
```

5. Update Data in the Table

Example: Update the age of the student named 'Bob'

```
UPDATE Students
SET age = 23
WHERE name = 'Bob';
```

6. View All Data

```
SELECT * FROM Students;
```

Write a My SQL queries to implement the subqueries in the My SQL command line client

Sample Tables Setup

```
CREATE DATABASE CompanyDB;
USE CompanyDB;
-- Employees table
CREATE TABLE Employees (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    department_id INT,
    salary INT
);
-- Departments table
CREATE TABLE Departments (
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    dept_name VARCHAR(100)
);
-- Insert data into Departments
INSERT INTO Departments (dept_name)
VALUES ('HR'), ('Engineering'), ('Marketing');
-- Insert data into Employees
INSERT INTO Employees (name, department_id, salary)
VALUES
('Alice', 2, 60000),
('Bob', 1, 40000),
('Charlie', 2, 80000),
('David', 3, 50000),
('Eve', 2, 70000);
```

Subqueries

1. Subquery in `WHERE` Clause

Get employees who earn more than the **average salary**:

```
SELECT name, salary
FROM Employees
WHERE salary > (
    SELECT AVG(salary)
    FROM Employees
);
```

2. Subquery in `FROM` Clause

Get the **average salary** by department using a subquery:

```
SELECT dept_name, avg_salary

FROM (

    SELECT department_id, AVG(salary) AS avg_salary

    FROM Employees

    GROUP BY department_id

) AS DeptAvg

JOIN Departments ON Departments.id = DeptAvg.department_id;
```

3. Subquery in SELECT Clause

Show each employee along with their **department name**:

```
SELECT

    name,

    (SELECT dept_name FROM Departments WHERE Departments.id =
Employees.department_id) AS department

FROM Employees;
```

4. Subquery with IN Operator

Find names of employees who work in **Engineering or Marketing**:

```
SELECT name

FROM Employees

WHERE department_id IN (

    SELECT id

    FROM Departments

    WHERE dept_name IN ('Engineering', 'Marketing')

);
```

5. Correlated Subquery

Find employees who earn more than the **average salary in their own department**:

```
SELECT name, salary

FROM Employees E

WHERE salary > (

    SELECT AVG(salary)

    FROM Employees

    WHERE department_id = E.department_id

);
```

Write a My SQL program to create the script files in the My SQL workbench

Step-by-Step Guide to Create Script Files in MySQL Workbench

Step 1: Open MySQL Workbench

- Launch **MySQL Workbench**.
- Connect to your **MySQL Server** using the username/password.

Step 2: Create a New SQL Script File

- Click on **File → New SQL Tab** or press Ctrl + T.
- This opens a blank SQL editor window.

Step 3: Write SQL Script

Enter the SQL code you want to execute.
Example Script (Database + Table + Insert):

```
-- Create a new database

CREATE DATABASE IF NOT EXISTS SchoolDB;

-- Use the database

USE SchoolDB;

-- Create a table
```

```
CREATE TABLE Students (

    student_id INT AUTO_INCREMENT PRIMARY KEY,

    name VARCHAR(100),

    grade INT

);

-- Insert sample data

INSERT INTO Students (name, grade)

VALUES

('Alice', 90),

('Bob', 85),

('Charlie', 78);
```

Step 4: Save the Script File

- Click **File → Save Script As** or press Ctrl + S.
- Save the file with a .sql extension, e.g., school_script.sql.

Step 5: Execute the Script

- To **run the entire script**, click the **lightning bolt icon** or press Ctrl + Shift + Enter.

Write a My SQL program to create a database directory in Project and initialize a database. sql file to integrate the database into API

Create a **database directory** in a project
Add a **database.sql** file with schema
Connect and **integrate the DB into an API**

Project Structure

```
my-api-project/

|

├── database/
```

```
|   └── database.sql      ← MySQL script (CREATE DB & TABLE)
|
├── db.js                 ← DB connection code
├── app.js                ← Express API server
├── package.json
```

```
-- Create Database
CREATE DATABASE IF NOT EXISTS api_project;
-- Use Database
USE api_project;
-- Create a Table
CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE
);
-- Insert Sample Data
INSERT INTO users (name, email)
VALUES
('Alice', 'alice@example.com'),
('Bob', 'bob@example.com');
```

You can execute this file using MySQL Workbench or command line:

```
mysql -u root -p < database/database.sql
```

3. db.js – MySQL Connection File

```
const mysql = require('mysql2');
```

```
const db = mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: '',       // Add your MySQL password
    database: 'api_project'
});
db.connect((err) => {
    if (err) {
        console.error('Database connection failed:', err.message);
    } else {
        console.log('Connected to MySQL database');
    }
});
module.exports = db;
```

4.app.js – Express API Using MySQL

```
const express = require('express');
const db = require('./db');
const app = express();
app.use(express.json());
// GET all users
app.get('/users', (req, res) => {
    db.query('SELECT * FROM users', (err, results) => {
        if (err) return res.status(500).send(err.message);
        res.json(results);
```

```
    });
});


// POST add user
app.post('/users', (req, res) => {
    const { name, email } = req.body;
    db.query('INSERT INTO users (name, email) VALUES (?, ?)', [name,
email], (err, result) => {
        if (err) return res.status(500).send(err.message);
        res.status(201).json({ id: result.insertId, name, email });
    });
});
// Server listen
app.listen(3000, () => {
    console.log('Server running at http://localhost:3000');
});
```

5. Install Dependencies

```
npm init -y
npm install express mysql2
```

6.Run the API

```
node app.js
```

Team Collaboration Using GitHub

o Form groups of 2–3 students

o Create a shared GitHub repo

o Assign tasks and work in branches

o Use Issues, Pull Requests, and Code Reviews

o Document code with README.md

- Branches
- Issues
- Pull Requests & Code Reviews
- README.md documentation

1. Create a Shared GitHub Repository

➢ **Steps:**

1. Go to https://github.com
2. Click **+ > New repository**
3. Fill in:
    - **Repository name** (e.g., college-management-api)
    - **Description**
    - Set to **Public** (or **Private** if team-specific)
    - Check Initialize with a README
4. Click **Create repository**
5. **Add collaborators**:
    - Go to Settings > Collaborators > Invite by GitHub username

2. Clone the Repo Locally

```
git clone https://github.com/your-username/college-management-api.git

cd college-management-api
```

3. Create and Use Branches for Task Assignments

Each member creates a branch for their assigned task:

```
git checkout -b login-feature
```

After work is done:

```
git add .

git commit -m "Add login API"
```

```
git push origin login-feature
```

Use GitHub Issues to Assign Work

1. Go to **Issues** tab in repo
   Click **New Issue**
2. Title it (e.g., “Add login feature”)
3. Assign it to a team member
4. Label it (e.g., feature, bug, enhancement)

5. Create Pull Requests (PRs)

When a task branch is ready:

1. Click **Compare & pull request** on GitHub
2. Add a clear title and description
3. Link the related issue (e.g., “Closes #1”)
4. Assign a **reviewer**

   Team members **review the PR**, request changes, or **approve**

After approval:

Merge the PR into `main`

6. Document the Project using README.md

In the root folder, edit or create a README.md:

````
# College Management API

This project is a Node.js + Express.js backend with MySQL integration for managing college modules like login, attendance, etc.

## Setup Instructions

```bash

git clone https://github.com/your-username/college-management-api.git

cd college-management-api

npm install
````
````

```
node app.js
```